

교과목 3 : 초거대언어모델(LLM)

단원 2 : 자연어 딥러닝

DAY3. 자연어 처리를 위한 언어 모델

목 차

1. BERT 모델의 개념과 구조		3
2. BERT 활용 자연어 이진 분류		5
3. BERT 활용 자연어 다중 분류		11
4. BERT 활용 감성 분류		14

1. BERT 모델의 개념과 구조

BERT 란?

- 구글이 **2018 년에 공개한 사전 훈련된 언어 모델**
- 등장과 동시에 수많은 자연어 처리 영역과 다양한 문제에서 최고의 성능을 보임
- 자연어 처리 분야에 한 획을 그은 모델로 평가받는 유용한 알고리즘
- **33 억 단어에 대해 약 4 일간 학습시킨 언어 모델**
 - 위키피디아의 **25 억 개 단어**와 북코퍼스(BookCorpus)의 **8 억 개 레이블이 없는 데이터**를 이용하여 사전 훈련됨
 - **레이블이 없는 데이터**로 먼저 학습한 후, 레이블이 있는 데이터에서 추가 학습하여 높은 성능을 얻음
 - **미세 조정(Fine-Tuning)** 과정에서 하이퍼파라미터를 다시 조정함
- **튜닝(Tuning)** 하여 다양한 용도로 활용 가능
 - BERT 모델의 구조를 변경하거나 추가 학습을 수행
 - 영어를 잘하는 사람이 비슷한 다른 언어를 쉽게 배우는 것과 유사한 원리
 - 예: **BERT 의 지식을 활용한 스팸 메일 분류기**

BERT 의 구조

BERT-Base 와 BERT-Large

변수 설명

- **L** = 인코더(Encoder) 층의 개수
- **D** = d_{model} (은닉 벡터)의 크기
- **A** = Self-Attention Head 의 개수

BERT-Base

- **L** = 12
- **D** = 768
- **A** = 12
- 약 **1 억 1 천만(110M)** 개의 파라미터

BERT-Large

- **L** = 24
- **D** = 1024

- $A = 16$
- 약 3억 4천만(340M) 개의 파라미터

특징

- BERT-Large 는 BERT-Base 보다
 - Self-Attention Head 수가 많고
 - d_{model} 의 크기가 더 크며
 - 전체 파라미터 수도 훨씬 많다.
- **Hugging Face 홈페이지**에서 무료로 다운로드하여 사용할 수 있다.
- 약 110 여 개 언어에 대한 BERT 모델이 제공된다.
 - 한국어 BERT 모델도 제공된다.
- BERT 모델을 다운로드한 후 **미세 조정(Fine-Tuning)** 을 수행하여 다양한 응용 프로그램에 적용할 수 있다.
- 다운로드한 모델을 Fine-Tuning 하여 영화 리뷰 분류, 스팸 메일 분류 등 다양한 자연어 처리 응용 분야에 활용할 수 있음을 소개한다.

2. BERT 활용 자연어 이진 분류

1. BERT Binary Classification 모델 구현

1) 학습 및 평가 데이터 세트

(1) 데이터 세트 준비

- 영화평 데이터 세트 업로드 및 Pandas를 이용한 데이터 로드

```
import pandas as pd

dataset = pd.read_table("/content/mnt/MyDrive/ratings.txt")
dataset.columns = ['id', 'review', 'sentiment']
dataset.head()
```

예시 출력

id	review	sentiment
9976970	더빙이 실망스럽네요.	0
3819312	오버 연기조차 가볍지 않구나	1
10265843	너무재밌었다. 그래서 보는 것을 추천한다	1
9045019	솔직히 재미는 없다.. 평점 조정	0
6483659	배우의 익살스런 연기가 돋보였던 영화!	1

Scikit-learn을 이용한 Train/Test 분리

- Train : 80%
- Test : 20%
- stratify 파라미터를 이용한 층화 샘플링(Stratified Sampling)

```
from sklearn.model_selection import train_test_split

train_idx, test_idx, _ = train_test_split(
    dataset.index,
    test_size=0.2,
    stratify=dataset.sentiment
)
```

```
train_set = dataset.iloc[train_idx]
test_set = dataset.iloc[test_idx]

train_set.reset_index(drop=True, inplace=True)
```

Train / Validation 분리

- Train : 70%
- Validation : 30%

```
train_idx, valid_idx, _ = train_test_split(
    train_set.index,
    test_size=0.3,
    stratify=train_set.sentiment
)

valid_set = train_set.iloc[valid_idx]
train_set = train_set.iloc[train_idx]

train_set.shape
valid_set.shape
test_set.shape
```

결과

```
(84000, 3)
(36000, 3)
(30000, 3)
```

(2) 데이터 세트 클래스 정의

torch.utils.data.Dataset을 상속하여 구현
재정의해야 하는 함수

```
__init__()
__len__()
__getitem__()
```

각 함수 역할

init()

- Text
- Label
- Tokenizer
- 최대 Token 길이 설정

len()

- 전체 데이터 개수 반환

getitem()

특정 index의

- input ids
- attention mask
- label

반환

(3) Pre-trained Tokenizer 다운로드

Huggingface에서 다운로드

```
bert_model_name = "kykim/bert-kor-base"

tokenizer = BertTokenizerFast.from_pretrained(
    bert_model_name
)
```

(4) 데이터 세트 클래스 객체 생성

Train 데이터 세트

```
train_set_dataset = BertDataset(
    reviews=train_set.review.tolist(),
    sentiments=train_set.sentiment.tolist(),
    tokenizer=tokenizer,
)
```

Validation 데이터 세트

```
valid_set_dataset = BertDataset(  
    reviews=valid_set.review.tolist(),  
    sentiments=valid_set.sentiment.tolist(),  
    tokenizer=tokenizer,  
)
```

2) Pre-trained 모델 다운로드 및 Fine-tuning 설정

(1) Pre-trained 모델 다운로드

Huggingface에서 kykim/bert-kor-base 모델 다운로드

```
from transformers import BertForSequenceClassification  
  
model = BertForSequenceClassification.from_pretrained(  
    bert_model_name  
)
```

참고

한국어 모델 검색

<https://huggingface.co/models>

검색어

kor

(2) Fine-tuning 설정

Transfer Learning 전략

```
tl_strategy = 3
```

전략 1

모든 BERT Layer Freeze

```
for name, param in model.bert.named_parameters():  
    print(name)  
    param.requires_grad = False
```


전략 2

Pooler만 학습

```
for name, param in model.bert.named_parameters():
    if not name.startswith("pooler"):
        param.requires_grad = False
```

전략 3

Pooler와 마지막 Encoder Layer만 학습

```
for name, param in model.bert.named_parameters():

    if (
        not name.startswith("pooler")
        and "layer.23" not in name
    ):
        param.requires_grad = False
```

3) 모델 학습 및 평가

(1) Training 하이퍼파라미터 설정

설정 항목

- Output Directory
- Epoch
- Batch Size
- Warmup Step
- Weight Decay

```
training_args = TrainingArguments(
    output_dir=model_dir,
    num_train_epochs=1,
    per_device_train_batch_size=128,
    per_device_eval_batch_size=64,
    warmup_steps=500,
    weight_decay=0.01,
    save_strategy="epoch",
    evaluation_strategy="steps"
)
```

(2) Evaluation용 Test 데이터 준비

Test Dataset 생성

```
test_set_dataset = BertDataset(  
    reviews=test_set.review.tolist(),  
    sentiments=test_set.sentiment.tolist(),  
    tokenizer=tokenizer  
)
```

Predict 설정

```
training_args = TrainingArguments(  
    output_dir="./model",  
    do_predict=True  
)
```

(3) Predict 수행

Trainer 생성

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    compute_metrics=compute_metrics,  
)
```

예측 수행

```
trainer.predict(test_set_dataset)
```

3. BERT 활용 자연어 다중 분류

1. BERT Multi Classification 모델 구현

1) 학습 및 평가 데이터 세트

(1) 데이터 세트 준비

News 데이터 로드

```
import pandas as pd

dataset = pd.read_csv(
    "/content/mnt/MyDrive/news.csv"
)
```

예시

```
dataset[dataset.category == 7].head()
```

출처

공공데이터포털
한국언론진흥재단
뉴스빅데이터 메타데이터
올림픽
2021
<https://www.data.go.kr>

Label(Category)별 데이터 개수

```
dataset.category.value_counts()
```

예시

```
7    200
6    200
5    200
4    200
3    200
2    200
```

1	200
0	200

데이터 크기

train_set.shape
valid_set.shape
test_set.shape

결과

(1024, 3)
(256, 3)
(320, 3)

(2) 데이터 세트 클래스 정의

News와 Category를 사용하도록 수정

<pre>class BertDataset(torch.utils.data.Dataset): def __init__(self, news, category, tokenizer): self.news = news self.category = category self.tokenizer = tokenizer self.max_len = tokenizer.model_max_length</pre>
--

2) Pre-trained 모델 다운로드 및 Fine-tuning 설정

(1) Pre-trained 모델 다운로드

<pre>from transformers import BertForSequenceClassification num_labels = 8 model = BertForSequenceClassification.from_pretrained(bert_model_name,</pre>

```
num_labels=num_labels
)
```

- num_labels = 8은 분류할 Category(Class)의 개수를 의미한다.

(2) Training 하이퍼파라미터 설정

```
training_args = TrainingArguments(
    output_dir=model_dir,
    num_train_epochs=1,
    per_device_train_batch_size=128,
    per_device_eval_batch_size=64,
    warmup_steps=500,
    weight_decay=0.01,
    save_strategy="epoch",
    evaluation_strategy="steps"
)
```

4. BERT 활용 감성 분류

1. BERT 모델 Fine-tuning

1) Huggingface

(1) Huggingface 란?

Pre-trained 모델을 공유하기 위한 모델 공유 플랫폼(Model Hub)

→ 모델 다운로드 및 Fine-tuning 을 이용하여 활용 가능

- 언어 모델 Fine-tuning 목적에 따라 모델 공유

※ 출처 : <https://huggingface.co/models>

- 딥러닝 프레임워크 라이브러리, 데이터 세트, 언어별 모델 공유

※ 출처 : <https://huggingface.co/models>

- 모델 검색 및 모델명을 이용한 다운로드

※ 출처 : <https://huggingface.co/models>

2) Huggingface Transformer

(1) Huggingface Transformer 란?

- ① 자연어 처리를 위한 범용 아키텍처 제공
- ② BERT 계열, GPT 계열 등의 다양한 언어 모델 아키텍처 제공
- ③ PyTorch, TensorFlow 등 다양한 프레임워크 지원 가능
- ④ Python 패키지로 제공되어 설치 및 사용 용이

3) Huggingface 와 Transformer

(1) Huggingface Transformer 설치

```
!pip install transformers
```

(2) Huggingface Transformer 모델 다운로드

Down Stream TASK 에 따른 클래스 import

```
from transformers import BertForSequenceClassification
```

참고) Down Stream TASK 란?

모델 Fine-tuning 을 통해 수행하고자 하는 작업
예)

- Text Classification
- Question Answering
- Summarization 등

다운로드할 Pre-train 모델명 지정

```
bert_model_name = "bert-base-uncased"  
from_pretrained() 함수를 이용한 모델 다운로드  
model = BertForSequenceClassification.from_pretrained(  
    bert_model_name  
)
```

(3) 다운로드 모델 구조

다운로드 모델 구조 확인

```
for name, param in model.bert.named_parameters():  
    print(name)
```

BERT Embedding Layer

```
embeddings.word_embeddings.weight  
embeddings.position_embeddings.weight  
embeddings.token_type_embeddings.weight  
embeddings.LayerNorm.weight  
embeddings.LayerNorm.bias
```

BERT Encoder × 12

예시

```
encoder.layer.0.attention.self.query.weight  
encoder.layer.0.attention.self.query.bias  
encoder.layer.0.attention.self.key.weight  
encoder.layer.0.attention.self.key.bias  
encoder.layer.0.attention.self.value.weight  
...
```

```
encoder.layer.0.output.dense.weight  
encoder.layer.0.output.dense.bias
```

Classification Layer

```
pooler.dense.weight  
pooler.dense.bias
```

다운로드 모델 파라미터 출력

```
for name, param in model.bert.named_parameters():  
    print(name, param)
```

(각 Layer 의 Weight 와 Bias Tensor 가 출력됨)

(4) 모델 Fine-tuning 수행

(Trainable / Non-trainable 설정)

```
requires_grad = True
```

- 해당 Layer 의 파라미터들을 학습을 통해 변경

```
requires_grad = False
```

- 해당 Layer 의 파라미터를 Frozen 상태로 두어 학습을 통해 변경하지 않음

예시

```
if tl_strategy == 1:  
    for name, param in model.bert.named_parameters():  
        param.requires_grad = False  
  
elif tl_strategy == 2:  
    for name, param in model.bert.named_parameters():  
        if not name.startswith("pooler"):  
            param.requires_grad = False  
  
elif tl_strategy == 3:  
    for name, param in model.bert.named_parameters():  
        if (not name.startswith("pooler")) and ₩  
            "layer.23" not in name:  
            param.requires_grad = False
```


2. BERT 모델 감성분석

1) 학습 및 평가 데이터 세트

(1) 데이터 세트 준비

IMDB 데이터 세트 업로드 및 Pandas 를 이용한 데이터 로드

```
import pandas as pd

dataset = pd.read_csv(
    "/content/mnt/MyDrive/IMDB Dataset.csv"
)

dataset.head()
```

데이터 예시

```
review
sentiment
```

Scikit-learn 을 이용한

Train(80%)

Test(20%)

데이터 세트 분리

층화 샘플링(Stratified Sampling)

```
from sklearn.model_selection import train_test_split

train_idx, test_idx = train_test_split(
    dataset.index,
    test_size=0.2
)

train_set = dataset.iloc[train_idx]
test_set = dataset.iloc[test_idx]

train_set.reset_index(drop=True, inplace=True)
```

Train 데이터 세트를 이용하여

Train(70%)

Validation(30%)

데이터 세트 분리

층화 샘플링(Stratified Sampling)

```
train_idx, valid_idx = train_test_split(
    train_set.index,
    test_size=0.3
)

valid_set = train_set.iloc[valid_idx]
train_set = train_set.iloc[train_idx]

train_set.shape
valid_set.shape
test_set.shape
```

예시 출력

```
(84000, 3)
(36000, 3)
(30000, 3)
```

(2) 데이터 세트 클래스 정의

torch.utils.data.Dataset 상속

재정의해야 하는 함수

```
__init__()
__len__()
__getitem__()
```

각 함수 역할

init()

- Text
- Label
- Tokenizer
- 최대 Token 수 지정

len()

- 전체 데이터 개수 반환

getitem()

- 입력 Text ID 반환
- Attention Mask 반환
- Label 반환

2) Pre-trained 모델 다운로드 및 Fine-tuning 설정

(1) Pre-trained Tokenizer 다운로드

Huggingface로부터 Tokenizer 다운로드

```
from transformers import BertTokenizerFast

bert_token_model = "bert-base-uncased"

bert_model_name = bert_token_model

tokenizer = BertTokenizerFast.from_pretrained(
    bert_token_model
)
```

주의

- Pre-trained 모델과 호환 가능한 Tokenizer 사용

Train 데이터와 Validation 데이터를 이용한 객체 생성

```
train_set_dataset = BertDataset(
    reviews=train_set.review.tolist(),
    sentiments=train_set.sentiment.tolist(),
    tokenizer=tokenizer
)

valid_set_dataset = BertDataset(
    reviews=valid_set.review.tolist(),
    sentiments=valid_set.sentiment.tolist(),
    tokenizer=tokenizer
)
```

(2) Pre-trained 모델 다운로드

```
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    bert_model_name
)
```

(3) Fine-tuning 설정

```
tl_strategy = 3
if tl_strategy == 1:
    ...
elif tl_strategy == 2:
    ...
elif tl_strategy == 3:
    ...
```

tl_strategy 를 이용하여

```
param.requires_grad=False
```

범위를 설정한다.

3) 모델 학습 및 평가

(1) Training 을 위한 하이퍼 파라미터 설정

- Output 디렉터리
- Epoch
- Batch Size
- Warmup
- Weight Decay

등 설정

예시

```
training_args = TrainingArguments(

    output_dir=model_dir,
    num_train_epochs=1,
```

```

        per_device_train_batch_size=128,
        per_device_eval_batch_size=64,
        warmup_steps=500,
        weight_decay=0.01,

        save_strategy="epoch",
        evaluation_strategy="steps"
    )

test_set_dataset = BertDataset(
    reviews=test_set.review.tolist(),
    sentiments=test_set.sentiment.tolist(),
    tokenizer=tokenizer
)

```

(2) Evaluation 을 위한 설정

Predict 수행을 위한 파라미터 설정

```

training_args = TrainingArguments(

    output_dir="./model",

    do_predict=True
)

```

(3) Predict 수행

```

trainer = Trainer(
    model=model,
    args=training_args,
    compute_metrics=compute_metrics
)

trainer.predict(test_set_dataset)

```

trainer.predict() 함수를 이용하여 Test 데이터에 대한 예측(Evaluation)을 수행한다.